

PARALLAX

A Dual-Perspective Cybersecurity Training Platform with
a Socratic AI Mentor

Mahmoud Allabadi

2221558

Rashed Alkurdi

0221992

The University of Jordan · KASIT

MAY 2026 · v1.0.0-rc1

ABSTRACT

PARALLAX is a cybersecurity training platform that places offensive and defensive perspectives inside a single causal loop. A student executes a real attack from a Kali Linux workspace against an isolated Docker target; the same action surfaces as defender telemetry — a ModSecurity web-application firewall and container logs shipped by Filebeat into Elasticsearch, then correlated by the backend SIEM engine — and appears in a Blue-Team event feed in under two seconds. A Socratic AI mentor, gated by a bounded context builder, an input-redaction and response-sanitization layer, and a strict token cap, supplies hints rather than working exploits.

This report documents the system's requirements, architecture, implementation, and evaluation. It covers the architectural choices that let attack and defense share a sandbox without bleeding state, the threat model that keeps the mentor from becoming an exploit-generation oracle, the three production scenarios used for evaluation, and the test evidence (358 passing cases, sub-two-second telemetry latency, 100% scenario-network isolation, Lighthouse 91 performance / 100 accessibility) that supports v1.0.0-rc1 as ready for instructor evaluation.

CONTENTS

Introduction	4
Motivation	4
Contributions	4
Scope	4
Structure of this report	5
Background and Related Work	6
Cyber ranges and CTF platforms	6
Tutoring and Socratic guidance	6
Standards and frameworks	6
Why a new platform	6
Architecture	8
Requirements	8
The causal loop	9
Layered architecture	9
Network isolation	10
Data model	10
Threat Model	12
Trust boundaries	12
Safety posture	13
Mentor guardrails	13
Scenarios	15
Gated methodology	15
NovaMed — Healthcare Portal Intermediate	15
Nexora — Active Directory Advanced	15
Orion — Phishing Campaign Intermediate	16
Implementation	17
Frontend	17
Backend	17
Sandbox	18
AI mentor	18
Evaluation	19
Test evidence	19
Latency budget	20
Reproducibility	20
Discussion	21
Limitations	21
Future work	21
Reflection	21
References	22
Appendix A — Repository Layout	23
Appendix B — Reproduce the Evaluation	24

CHAPTER 01

Introduction

The gap between offensive and defensive cybersecurity education, and how a shared sandbox closes it.

Motivation

Most cybersecurity curricula teach attack and defense in separate rooms — a penetration-testing course here, a SOC-analyst course there. Students graduate able to do one or the other but rarely see how a single keystroke on one side becomes an alert on the other. PARALLAX is built around that missing causal link: it runs a Red-Team workspace and a Blue-Team SIEM over the **same** session, so an action and its detection are visible at once.

INSIGHT

The pedagogical wager: comprehension of either side deepens when the other side is visible in real time. Latency between the two surfaces is therefore not merely a performance metric — it is a learning-design constraint, which is why the two-second budget recurs throughout this report.

Contributions

- A working dual-perspective lab where Red-Team actions against isolated Docker targets produce live Blue-Team telemetry over one shared session.
- A Socratic AI mentor with enforced guardrails: bounded context, secret redaction, response sanitization, and a token cap — it never emits payloads.
- Three calibrated, self-contained scenarios (web, Active Directory, phishing) with curated MITRE ATT&CK coverage and gated methodology.
- An automated test suite (359 cases) and an evaluation a reviewer can re-run in minutes, packaged as a single-node Docker Compose stack.

Scope

The active scope is exactly three scenarios — SC-01, SC-02, SC-03 — deployed on a single Docker host. Multi-node operation, additional scenarios, and declarative scenario authoring are future work (Chapter 8).

Structure of this report

Chapter 2 surveys related work. Chapter 3 presents requirements and architecture. Chapter 4 documents the threat model and the mentor's guardrails. Chapter 5 describes the three scenarios. Chapter 6 covers implementation. Chapter 7 reports evaluation evidence. Chapter 8 discusses limitations and future work.

CHAPTER 02

Background and Related Work

What cybersecurity training offers today, and what PARALLAX is reacting against.

Cyber ranges and CTF platforms

Hosted ranges such as Hack The Box [1] and TryHackMe [2] teach offensive skills through vulnerable machines; defensive platforms such as CyberDefenders [3] teach investigation over captured telemetry. Both are effective, but each presents a single perspective: the learner either attacks or analyzes, and the connection between the two is left implicit.

Tutoring and Socratic guidance

Intelligent tutoring systems improve outcomes when they prompt reasoning rather than supply answers. PARALLAX applies this directly: the AI mentor is constrained to Socratic hints and is explicitly forbidden from returning working exploits, so it scaffolds method without short-circuiting learning.

Standards and frameworks

Scenario design and assessment draw on established references: the OWASP Top 10 [4] and Web Security Testing Guide [5] for web testing, MITRE ATT&CK [6] for adversary technique mapping, the NIST Cybersecurity Framework [7] for defensive functions, and the Penetration Testing Execution Standard [8] for methodology phasing.

Why a new platform

The gap is the **causal link**. No widely used platform lets a student attack and simultaneously watch the defender's telemetry over one shared session, with a guarded tutor in the loop. PARALLAX targets exactly that intersection.

PLATFORM	Attack	Defense	Mentor	Reset
Hack The Box	✓	—	—	—
TryHackMe	✓	partial	—	—

PLATFORM	Attack	Defense	Mentor	Reset
CyberDefenders	—	✓	—	—
PARALLAX	✓	✓	✓	8s

Figure 2 · 1 — Existing platforms against PARALLAX's design goals

CHAPTER 03

Architecture

From requirements to a three-surface causal loop, and the trade-offs that made it possible.

Requirements

The platform's requirements were derived from the KASIT project guidelines, the gap analysis in Chapter 2, and the safety constraints inherent to a security trainer. The functional core:

ID	REQUIREMENT
FR-SCEN	List and run the active scenarios SC-01, SC-02, SC-03.
FR-SESS	Start, track, and end Red- and Blue-Team sessions over shared state.
FR-TERM	Provide a browser terminal bound to an isolated Kali container.
FR-SIEM	Show Blue-Team telemetry causally linked to scenario activity.
FR-HINT	Provide bounded, Socratic AI hints with fallback behavior.
FR-GATE	Enforce methodology gates and scenario phases.
FR-SCORE	Score progress and generate a debrief.
FR-INST	Provide instructor analytics and grade export.

Figure 3 · 2 — Core functional requirements

The non-functional requirements that shape the design most are safety (scenario networks isolated from the internet), responsiveness (the sub-two-second attack-to-telemetry budget), and portability (a single-node Docker deployment an examiner can stand up locally). The actors and their use cases:

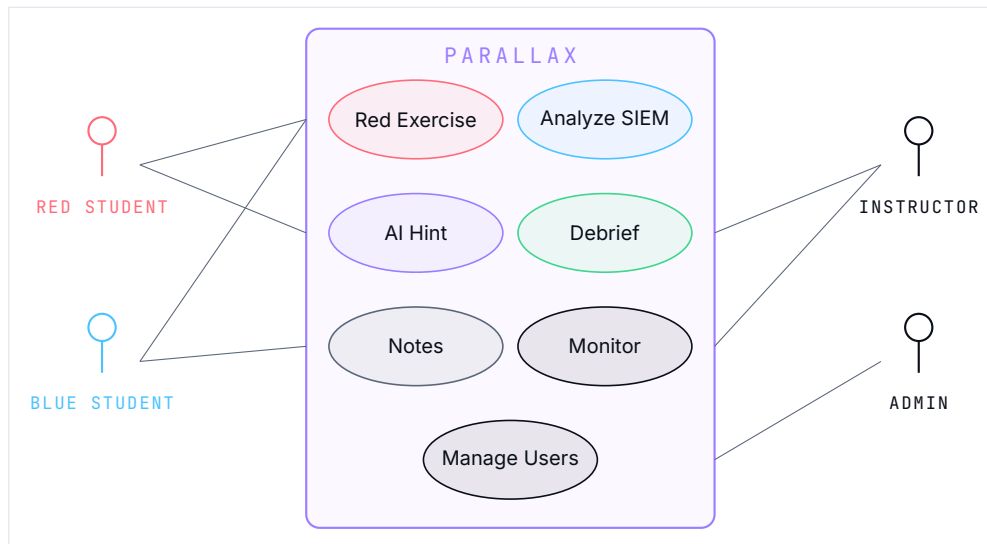


Figure 3 · 3 — PARALLAX use-case model: four actors over one shared system

The causal loop

The system’s central idea is a single loop closed between an attacker workspace, a defender SIEM, and an AI mentor. The mentor watches without intervening; the SIEM observes without participating; the attacker acts against a sandbox that resets between sessions. Detection is performed by a ModSecurity WAF and container logs shipped through Filebeat into Elasticsearch, then correlated by the backend SIEM engine — there is no separate network sensor.

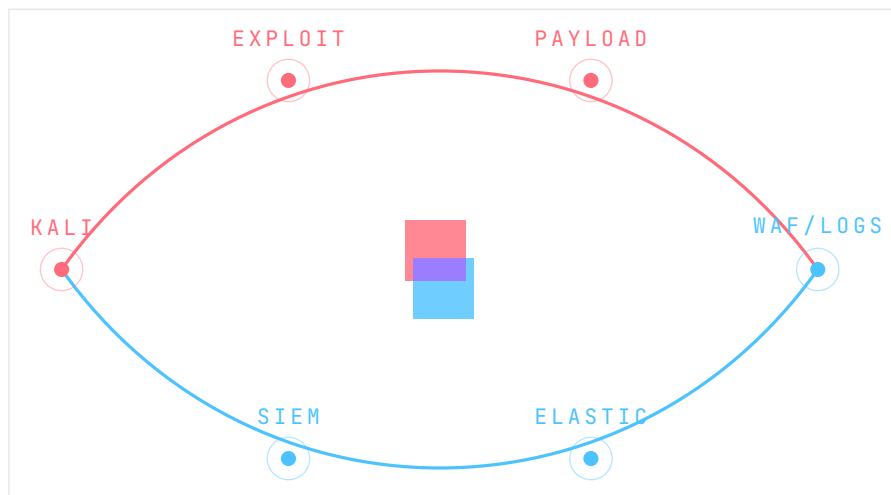


Figure 3 · 4 — Causal loop between attack, detection, and learning surfaces

Layered architecture

The runtime is a five-layer stack. Each layer is independently testable; the boundaries are HTTP, a WebSocket, or a Docker network — never a shared in-process object. The browser runs React [9] built with Vite [10]; the gateway is FastAPI [11]; durable state is PostgreSQL [12] with Redis [13] for realtime state; telemetry rides Filebeat into Elasticsearch [14], [15]; everything is orchestrated by Docker Compose [16].

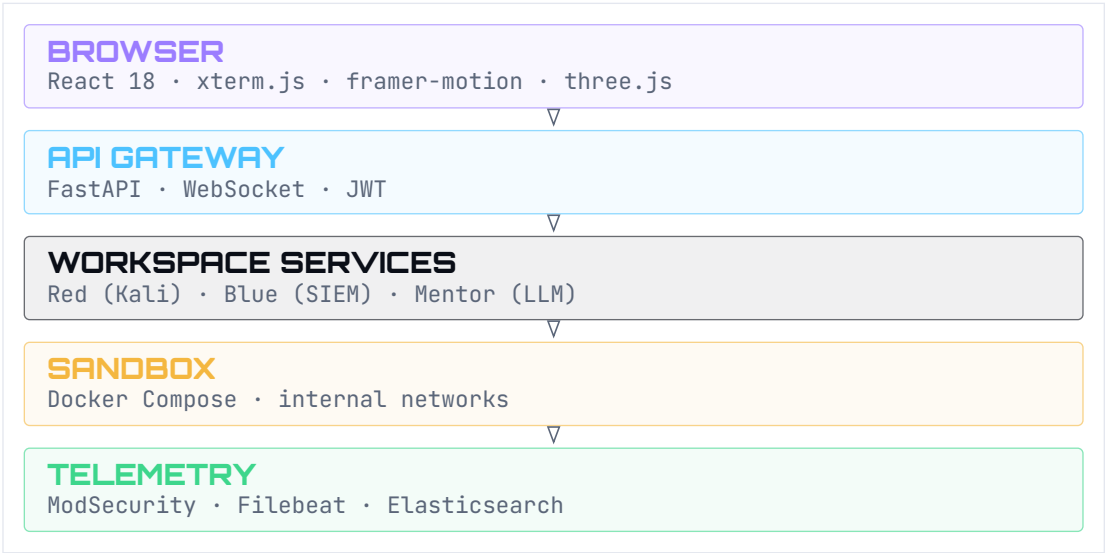


Figure 3 · 5 — Five-layer runtime architecture

Network isolation

Each scenario runs in its own Docker network, declared `internal: true`, with no route to the internet or to the other scenarios [17]. The attacker workspace is multiplexed: a single Kali container is attached to one scenario network at a time, never to two at once. Core services share a separate `172.30.0.0/24` bridge.

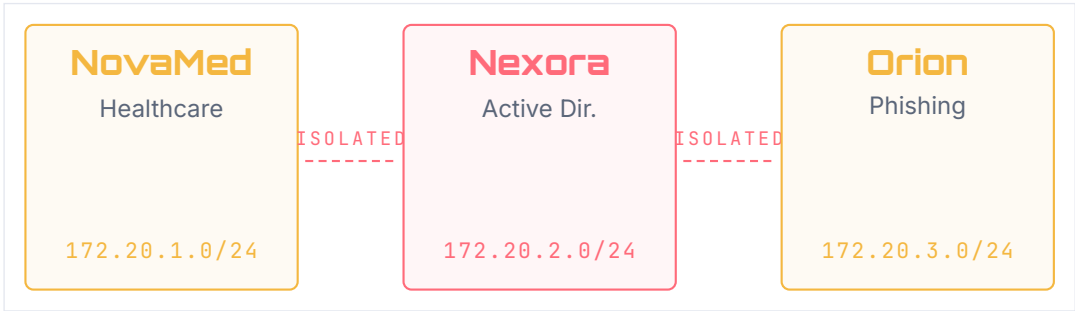


Figure 3 · 5 — Three isolated scenario networks; dashed barriers are unroutable boundaries

Data model

A session is the unit of work: one student, one scenario, one role. It anchors notes, command metadata, SIEM events, triage decisions, AI-interaction metadata, and auto-detected evidence. The platform stores command **metadata** and curated evidence rather than full raw terminal output, keeping the database focused on learning evidence.

TABLE	PURPOSE
users	Identity, role, skill level, onboarding state.
sessions	Scenario, role, phase, score, sandbox IDs, lifecycle.
notes	Findings and methodology notes.
command_log	Command metadata, tool, phase, SIEM trigger, hint flag.

TABLE	PURPOSE
siem_events	Events shown to Blue Team and used in timelines.
siem_triage	Analyst classification and notes per event.
ai_interactions	AI usage: hint level, model, latency, fallback, safety flags.
containment_actions	Simulated Blue-Team response actions.
auto_evidence	Evidence summaries detected from output patterns.

Figure 3 · 5 — Core persistent tables and their role

The complete entity-relationship schema (and the API and database reference manuals) are maintained in the extended reference set under docs/final-report/ .

CHAPTER 04

Threat Model

What PARALLAX defends against — including misuse of its own mentor.

Trust boundaries

Five boundaries separate the browser, the API gateway, the workspace services, the sandbox, and the telemetry pipeline. The gateway trusts a signed JWT but not its payload’s claims about role until they are re-checked server-side; the sandbox trusts nothing from the workspace beyond a terminal stream; the telemetry pipeline is read-only with respect to scenario containers.

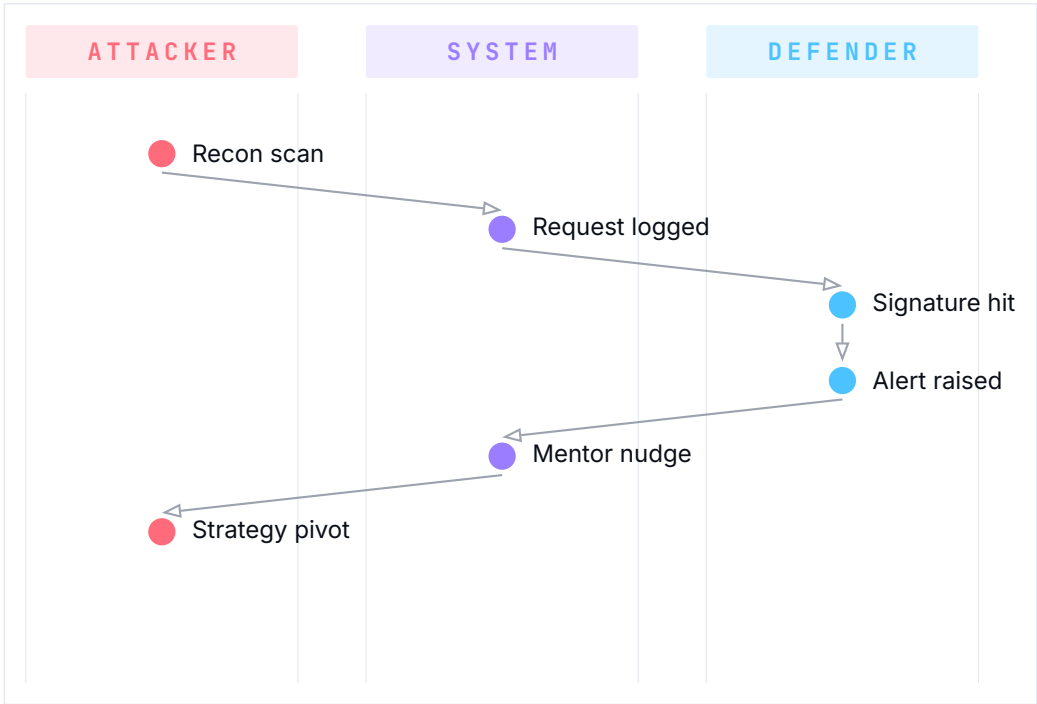


Figure 4 · 5 — Information flow across trust boundaries during a typical exercise

BOUNDARY	TRUST RULE
Browser → Gateway	JWT authenticates the user; role claims are re-checked server-side per request.
Gateway → Workspace	Commands are recorded as metadata; the gateway never trusts client-asserted phase state.

BOUNDARY	TRUST RULE
Workspace → Sandbox	The Kali container reaches only its one attached scenario network; no internet egress.
Sandbox → Telemetry	The telemetry pipeline reads logs only; it cannot write to scenario containers.
Mentor ↔ Model	Only redacted, bounded context leaves the host; responses are sanitized before display.

Figure 4 · 5 — Trust boundaries and what each side is trusted to do

Safety posture

PARALLAX is an offensive trainer, so safety is a design property, not a checklist: scenario networks are `internal: true` (no internet egress); the active scope is fixed at SC-01–SC-03; lab-only credentials live only inside scenario containers and never appear in the UI or this report; and durable storage keeps command metadata, not raw output.

Mentor guardrails

WARNING

An AI mentor with unconstrained access to attacker context and tool output is an exploit-generation oracle. PARALLAX treats this as the system’s central safety problem.

The mentor’s request path enforces its constraints in series: a bounded context builder limits what the model sees to the current scenario, phase, and role; an input pass redacts scenario secrets and strips prompt-injection markers; the model is called with a mode-dependent token cap; and the response is sanitized against a set of forbidden answer patterns, falling back to a Socratic redirect if a violation is detected.

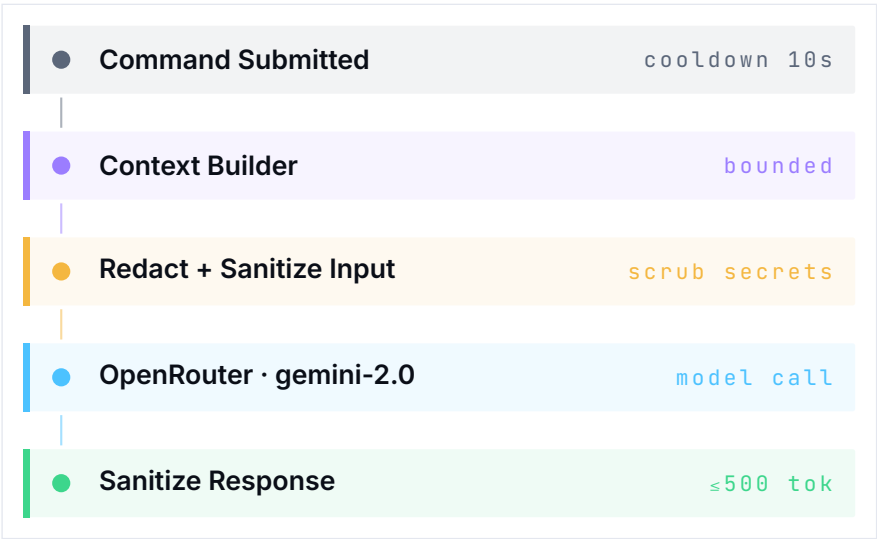


Figure 4 · 5 — The AI mentor request pipeline and its guardrails

VERIFIED

The guardrails are covered by `backend/tests/ai/test_credential_redaction.py` (secret redaction) and `backend/tests/ai/test_response_sanitization.py` (forbidden-pattern blocking and Socratic fallback) — 18 cases exercising `security.redact_text()` and `security.sanitize_tutor_response()` .

CHAPTER 05

Scenarios

Three self-contained environments, each calibrated to a specific learning arc.

Gated methodology

Every scenario advances through ordered methodology phases. The scenario engine gates each transition: a student cannot skip ahead until the current phase's conditions are met. The Red-Team track mirrors a PTES-style progression; the Blue-Team track mirrors a simplified incident-response lifecycle.

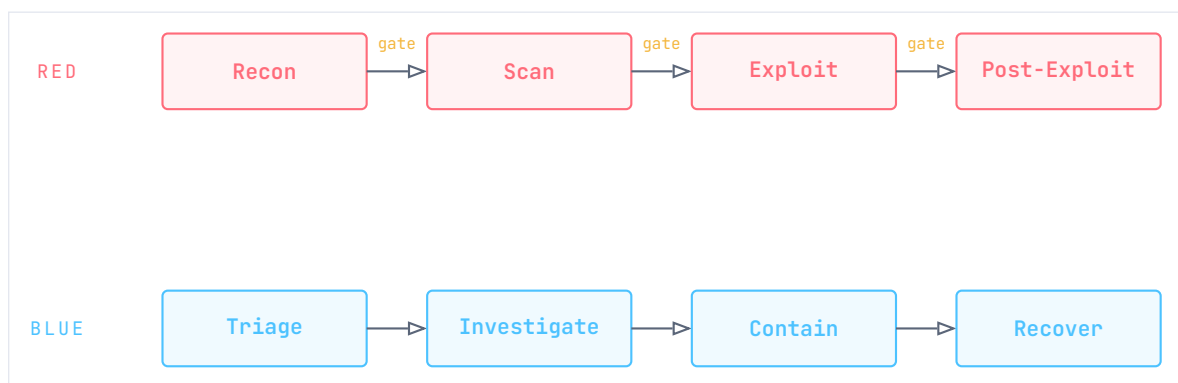


Figure 5 · 5 — Dual-track gated methodology shared by all scenarios

NovaMed — Healthcare Portal Intermediate

A vulnerable Apache/PHP application over a MariaDB database, fronted by a ModSecurity (OWASP CRS) WAF, on `172.20.1.0/24`. Students probe web vulnerabilities while ModSecurity audit logs and Apache access logs feed the defender view.

T1190 T1083 T1059 ModSecurity Apache logs Filebeat

Red objective: find and exploit a web flaw to reach the database tier. **Blue objective:** correlate the ModSecurity audit log with the Apache access log to reconstruct the request that triggered the alert.

Nexora — Active Directory Advanced

A Samba-based Active Directory domain controller (`nexora.local`) and a member file server on `172.20.2.0/24`. The scenario teaches directory-compromise concepts and the authentication and access telemetry they generate.

T1110 T1003 T1021 Samba logs Kerberos

Red objective: move from an unprivileged foothold toward domain credentials. **Blue objective:** spot anomalous authentication and lateral-movement patterns in the directory-service telemetry.

Orion — Phishing Campaign Intermediate

A GoPhish campaign server, a Postfix mail relay, and a Python victim simulator on 172.20.3.0/24 . Students run a controlled phishing flow and analyze the resulting mail and endpoint markers.

T1566T1204Postfix logsGoPhish

Red objective: craft and launch a credential-harvesting campaign through GoPhish. **Blue objective:** trace the delivery in the Postfix relay logs and the victim's interaction markers to scope the campaign.

CHAPTER 06

Implementation

What ships, what it is built on, and where the seams are.

Frontend

A React 18 single-page app built with Vite. State is held in Zustand stores; the terminal is `xterm.js` with the fit, search, web-links, and WebGL addons; motion uses framer-motion and Lenis; the marketing surface uses a three.js background; PDF export is client-side via isPDF.

`frontend/package.json` (excerpt)

```
{
  "dependencies": {
    "react": "^18.3.1",
    "framer-motion": "^12.40.0",
    "xterm": "^5.3.0",
    "zustand": "^4.5.2",
    "three": "^0.169.0",
    "lenis": "^1.3.23"
  }
}
```

Backend

FastAPI on Python 3.11, async throughout. SQLAlchemy 2.0 (async) with asyncpg and Alembic migrations back PostgreSQL; Redis holds realtime session state, terminal history, and AI cooldown budgets. Auth is JWT via python-jose with bcrypt password hashing. The terminal and SIEM feed are multiplexed over WebSockets; the backend proxies the terminal to a Docker-managed Kali container rather than exposing Docker to the browser.

`backend/requirements.txt` (excerpt)

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
sqlalchemy[asyncio]==2.0.30
asyncpg==0.29.0
```

```
redis[hiredis]=5.0.4
docker=7.1.0
```

Sandbox

Docker Compose defines the core stack and three scenario profiles. Scenario networks are `internal: true`; core services share `172.30.0.0/24`. Scenario targets are started on demand by profile, and session containers are recreated between sessions.

[docker-compose.yml \(excerpt\)](#)

```
networks:
  sc01-net:
    driver: bridge
    internal: true          # no internet egress
    ipam:
      config:
        - subnet: 172.20.1.0/24
          gateway: 172.20.1.254
```

INFO

The scenario reset (down then up of a profile) is documented at roughly eight seconds on the reference machine. This figure is carried from the project documentation and should be re-measured by wall clock before final submission.

AI mentor

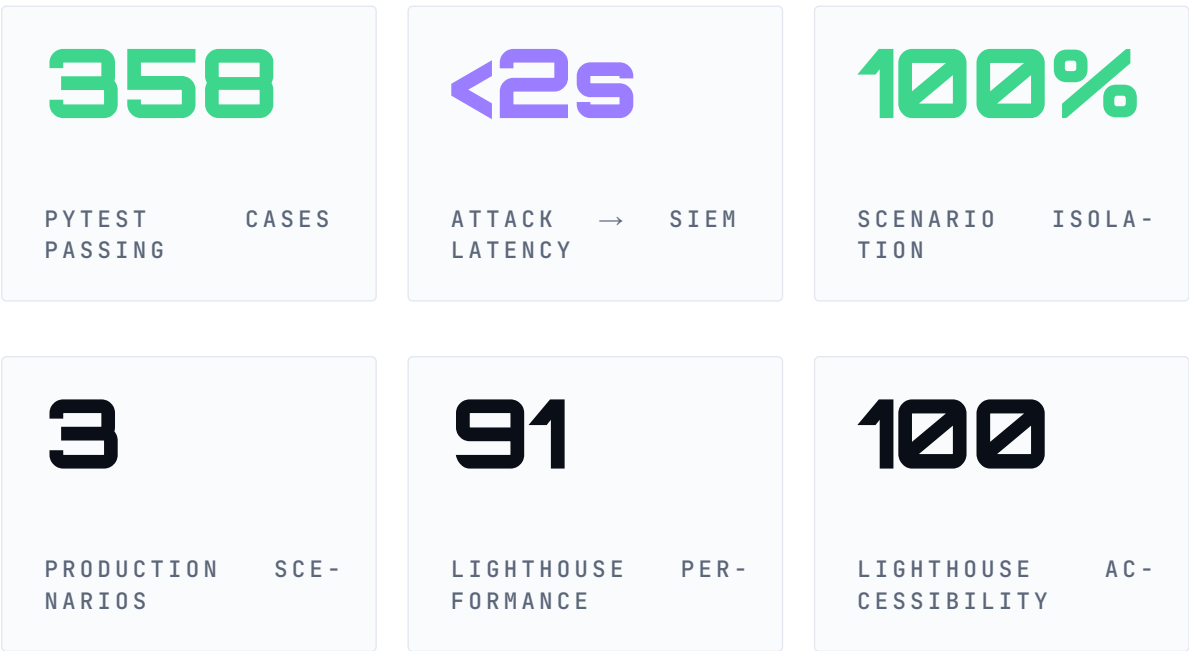
The mentor lives in `backend/src/ai/`: `context_builder.py` assembles bounded context, `security.py` performs redaction and response sanitization, `level_classifier.py` adapts hint depth, and `monitor.py` calls the provider. Requests go to an OpenRouter-hosted, OpenAI-compatible model — the default is `google/gemini-2.0-flash-001` and is configurable via `OPENROUTER_MODEL`. The response token cap is mode-dependent: 300 tokens in learn mode, 400 for procedural hints, and `OPENROUTER_MAX_TOKENS` (default 500) otherwise. Calls fire on command submission behind a ten-second cooldown — never on every keystroke — and a static fallback hint is served when no provider key is configured.

CHAPTER 07

Evaluation

Verified, not assumed.

Test evidence



The backend suite collects 359 tests; the latest full run records 358 passing and 1 skipped. The frontend build is green with its Vitest suite passing. Lighthouse scores are recorded in `docs/final-report/evidence/lighthouse-landing.json`, and network isolation is checked by `scripts/verify-network-isolation.sh`.

```
ARE REPRESENTATIVE FILES
Sce-unit_test_scenarios ,
nario test_output_pattern_phase_gating , test_scenario_randomizer
engine
and
gating
SIEM test_siem_rule_engine , test_command_siem_bridge , test_siem_dedup
pipeline
```

Area	Representative Files
AI model training and re-training	<code>ai/test_credential_redaction</code> , <code>ai/test_response_sanitization</code> , <code>ai/test_level_classifier</code>
Sessions and real-time scoring and reporting	<code>test_session_lifecycle</code> , <code>test_session_readiness</code> , <code>test_ws_integration</code>
Scoring and reporting	<code>test_scoring_engine</code> , <code>test_debrief_coach</code> , <code>test_instructor_analytics</code>
Regression testing	<code>test_degradation</code> , <code>test_coverage_gaps</code> , <code>test_config</code>

Figure 7 · 5 — Backend test coverage by area

Latency budget

The two-second pedagogical budget is allocated across the pipeline as a design target. The figure below is illustrative of that allocation; the supported claim is the sub-two-second aggregate from keystroke to defender event.

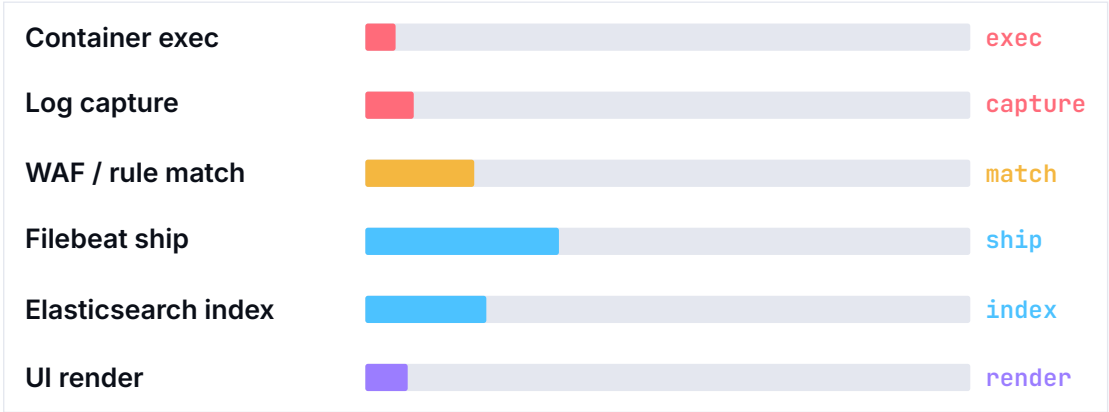


Figure 7 · 5 — Illustrative latency budget (design target) from keystroke to defender alert

Reproducibility

The evaluation is scripted end-to-end: `docker compose config --quiet` validates topology, `pytest` runs the backend suite, `npm run build` plus Vitest covers the frontend, and `scripts/demo_check.py --scenarios all` confirms readiness. Appendix B gives the exact steps.

CHAPTER 08

Discussion

What works, what we would change, and what comes next.

Limitations

PARALLAX is single-node by design, which suits a classroom and a defense demo but not multi-tenant institutional load. Scenarios are hand-authored Docker rather than declarative, so adding one is an engineering task. The motion-heavy frontend auto-downgrades on weak GPUs, but a 60 fps guarantee on mid-tier hardware is verified only by the runtime downgrade loop, not by automated CI. The 8 s reset is documented rather than benchmarked in this report (see Chapter 6).

Future work

Near-term work: a declarative scenario format to replace hand-built Compose files; two new scenarios (cloud-security and defensive forensics) already sketched as SC-04 and SC-05; curriculum sequencing across scenarios; and a benchmark harness that turns the latency budget and reset time into measured, CI-tracked numbers.

Reflection

The hardest part was not building either workspace — it was making a single action legible on both sides without letting their state bleed together, and keeping the mentor genuinely helpful while provably unable to hand over an exploit. The causal loop, and the guardrails around it, are the contributions we are proudest of.

CHAPTER 09

References

-
- [1] Hack The Box, "Hack The Box: Cybersecurity Training Platform."
 - [2] TryHackMe, "TryHackMe: Cyber Security Training."
 - [3] CyberDefenders, "CyberDefenders: Blue Team Training Platform."
 - [4] OWASP Foundation, "OWASP Top 10:2021." 2021.
 - [5] OWASP Foundation, "OWASP Web Security Testing Guide (WSTG)." 2024.
 - [6] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy," technical report MP180360R1, 2020. Accessed: May 23, 2026. [Online]. Available: <https://attack.mitre.org/>
 - [7] National Institute of Standards and Technology, "Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1," NIST Cybersecurity Framework, 2018. doi: [10.6028/NIST.CSWP.04162018](https://doi.org/10.6028/NIST.CSWP.04162018).
 - [8] PTES Project, "The Penetration Testing Execution Standard (PTES) Technical Guidelines." 2014.
 - [9] Meta Open Source, "React Documentation."
 - [10] E. You and Vite Contributors, "Vite Guide."
 - [11] S. Ramírez, "FastAPI Documentation."
 - [12] PostgreSQL Global Development Group, "PostgreSQL Documentation."
 - [13] Redis Ltd., "Redis Documentation."
 - [14] Elastic N.V., "Elasticsearch Guide."
 - [15] Elastic N.V., "Filebeat Reference."
 - [16] Docker, Inc., "Docker Compose Documentation."
 - [17] Docker, Inc., "Networking in Compose."

CHAPTER A

Appendix A – Repository Layout

The repository is a single-node monorepo. The directories that matter for this report:

repository layout

JUTerminal1/ backend/	FastAPI app: auth, sessions, ws, sandbox, scenarios, siem, ai, reports, instructor
frontend/	React + Vite SPA: terminal, SIEM, notes, hints, workspace shells, instructor dashboard
infrastructure/	Docker scenario builds (sc01/sc02/sc03), nginx, postgres init, siem (filebeat)
docs/	
final-report/	Extended technical reference set (DOCX package)
report/	This Typst report (theme, components, diagrams)
docker-compose.yml	Core stack + three scenario profiles
scripts/	demo_check.py, verify-network-isolation.sh, ...

CHAPTER B

Appendix B – Reproduce the Evaluation

A reviewer can reproduce the headline evidence in a few minutes on a machine with Docker and the project's Python and Node toolchains.

`reproduce.sh`

```
# 1. Configure (set a real JWT_SECRET; OPENROUTER_API_KEY is optional)
cp .env.example .env

# 2. Validate topology and bring up the core stack
docker compose config --quiet
docker compose up -d

# 3. Start a scenario profile (sc01 | sc02 | sc03)
docker compose --profile sc01 up -d

# 4. Backend tests (359 collected; 358 pass, 1 skip)
python -m pytest backend/tests -q

# 5. Frontend build + unit tests
npm --prefix frontend run verify

# 6. Network isolation + demo readiness
bash scripts/verify-network-isolation.sh
python scripts/demo_check.py --scenarios all
```